

Prédire le prix de revente de voitures d'occasion

Résumé : Ce projet met en œuvre un pipeline complet de Machine Learning pour prédire le prix de revente de voitures d'occasion à partir d'un dataset eBay Kleinanzeigen (371 528 annonces, marché allemand). Le pipeline couvre l'EDA, le preprocessing, l'ingénierie des features, la modélisation (12 algorithmes), la cross-validation, le tuning des hyperparamètres et le déploiement via Streamlit. Le modèle final (XGBoost) atteint un R^2 de **0.887** et un RMSE de **2 595 EUR**.

1. Présentation du Projet

1.1 Contexte et Objectif

Le marché de l'automobile d'occasion représente un enjeu économique majeur. L'estimation du prix de revente est complexe car elle dépend de nombreux facteurs : la marque, l'âge, le kilométrage, la puissance, le type de carrosserie, etc. Ce projet vise à construire un modèle prédictif fiable permettant d'estimer automatiquement le prix d'un véhicule d'occasion à partir de ses caractéristiques.

1.2 Dataset

Paramètre	Valeur
Source	eBay Kleinanzeigen (Allemagne)
Taille initiale	371 528 lignes × 21 colonnes
Variable cible	price (EUR)
Période	Annonces automobiles allemandes
Tâche ML	Régression supervisée

1.3 Variables Utilisées

Le modèle exploite 11 variables explicatives couvrant les dimensions numériques et catégorielles du véhicule :

Variables numériques	Variables catégorielles
kilometer – kilométrage (km)	brand – marque
powerPS – puissance (CV)	model – modèle
monthOfRegistration – mois	vehicleType – type de carrosserie
age_vehicule – âge calculé (2016 - année)	gearbox – boîte de vitesse
postalCode – code postal	fuelType – type de carburant
	notRepairedDamage – dommages non réparés

2. Analyse Exploratoire des Données (EDA)

2.1 Distribution de la Variable Cible

La distribution du prix brute est fortement asymétrique à droite (right-skewed), avec une concentration des annonces entre 500 et 5 000 EUR. Pour corriger cette asymétrie et améliorer les performances des modèles linéaires, une transformation logarithmique $\log(1 + \text{prix})$ est appliquée, ce qui rapproche la distribution d'une loi normale.

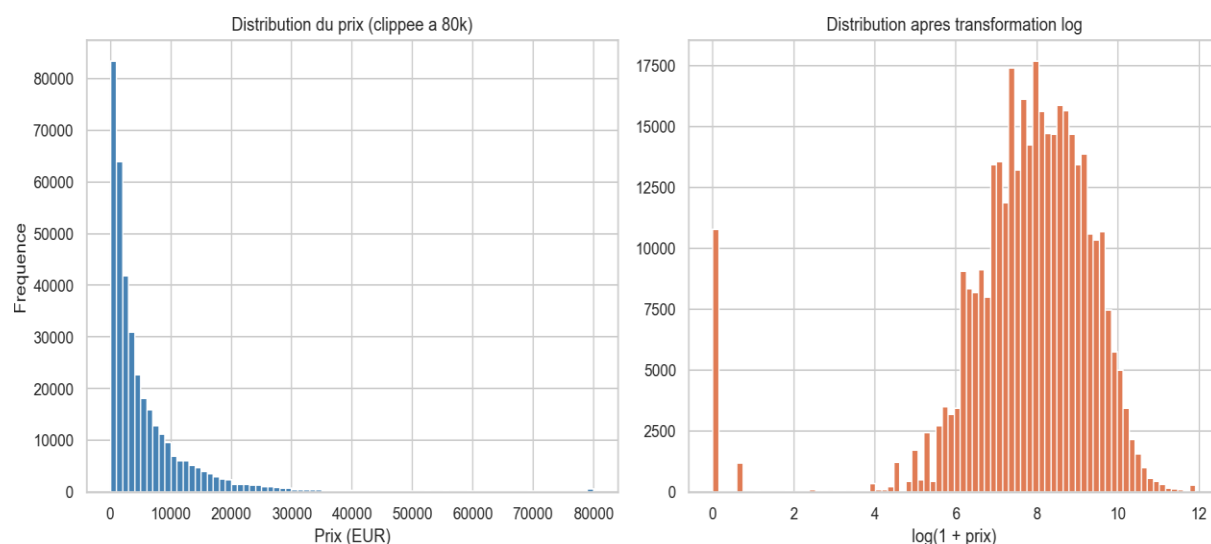


Figure 1 – Distribution du prix brut (gauche) et après transformation log (droite)

2.2 Distribution des Variables Catégorielles

L'analyse des variables catégorielles révèle une forte dominance de Volkswagen parmi les marques, de la carrosserie limousine/kleinwagen parmi les types de véhicules, et du moteur essence (benzin) parmi les types de carburant. La grande majorité des véhicules (85 %) n'ont pas de dommages non réparés.

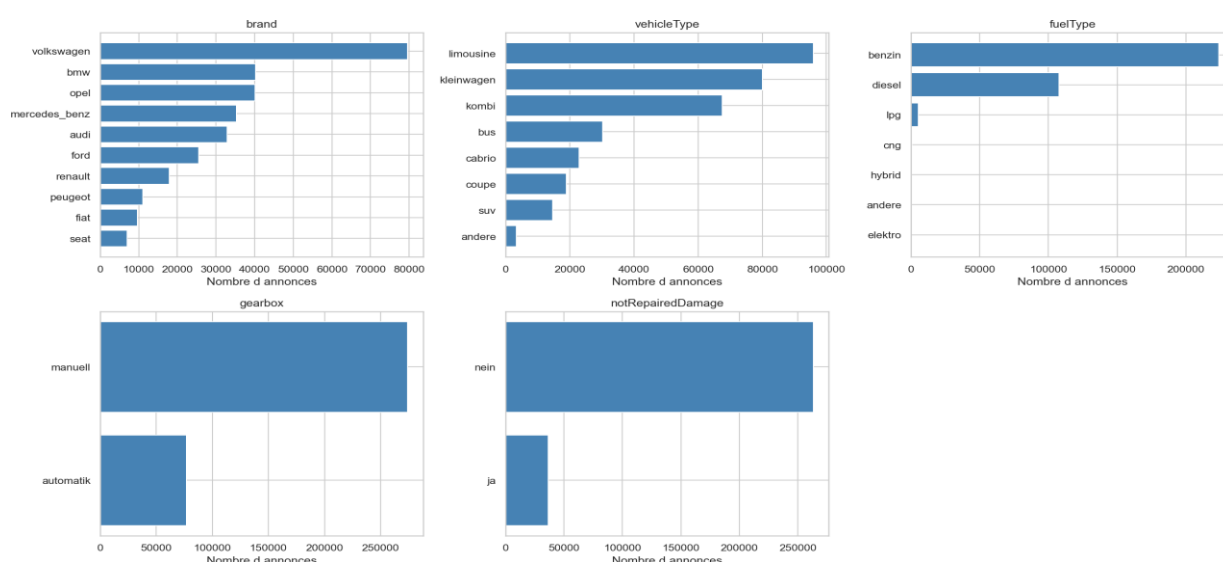


Figure 2 – Distribution des variables catégorielles (top 10 marques, types de véhicules, carburant, boîte, dommages)

2.3 Prix Médian par Marque

L'analyse du prix médian par marque met en évidence un écart significatif : Audi, BMW et Mercedes-Benz affichent les prix médians les plus élevés (autour de 5 000 EUR), tandis que Renault, Fiat et Opel se

positionnent en bas du classement (~1 500 EUR). Cette variable constitue donc un prédicteur important du prix.

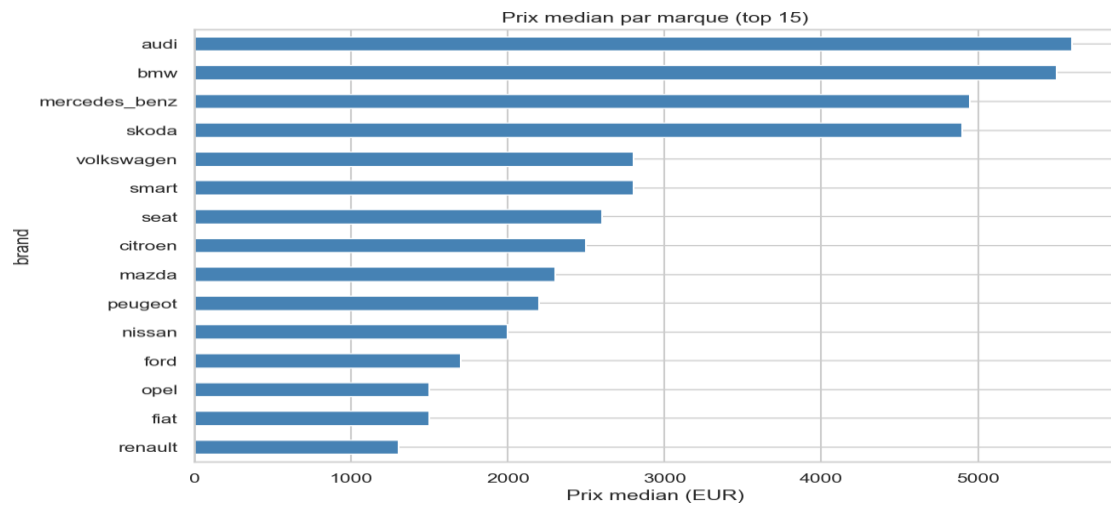


Figure 3 – Prix médian par marque (top 15)

2.4 Corrélations Numériques

Les scatter plots kilométrage vs prix et puissance vs prix confirment les intuitions métier : plus un véhicule a de kilométrage, moins il vaut ; à l'inverse, une puissance plus élevée est associée à un prix plus élevé. La variable age_vehicule (dérivée de l'année d'immatriculation) est la variable numérique la plus corrélée au prix.

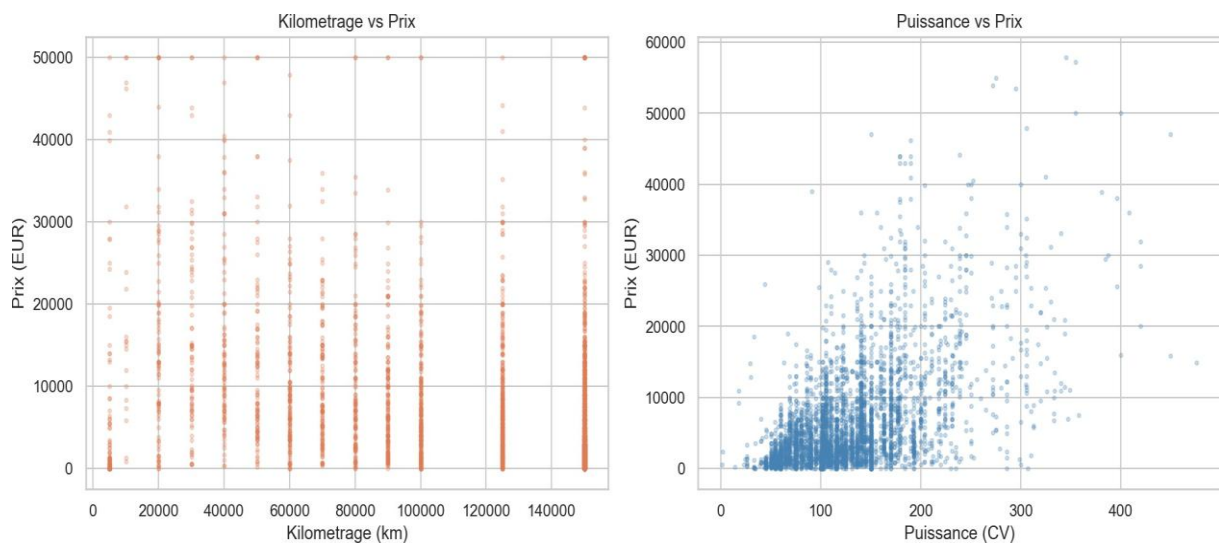


Figure 4 – Kilométrage vs Prix (gauche) et Puissance vs Prix (droite)

3. Preprocessing et Feature Engineering

3.1 Nettoyage des Données

Avant toute modélisation, plusieurs opérations de nettoyage sont appliquées pour assurer la qualité des données :

- **Filtrage des prix aberrants** : Suppression des annonces avec prix = 0 ou prix > 150 000 EUR
- **Filtrage des puissances** : Élimination des véhicules avec powerPS = 0 ou > 500 CV
- **Filtrage des années** : Conservation des véhicules immatriculés entre 1950 et 2016
- **Valeurs manquantes** : Imputation par la médiane pour les variables numériques, par le mode pour les catégorielles

3.2 Feature Engineering

Une nouvelle variable **age_vehicule** est créée à partir de l'année d'immatriculation : $\text{age_vehicule} = 2016 - \text{yearOfRegistration}$. Cette transformation convertit l'année (une variable ordinale peu informative) en un âge numérique directement interprétable par les modèles. La variable yearOfRegistration est ensuite supprimée du dataset.

3.3 Pipeline de Preprocessing

Un pipeline sklearn est construit pour garantir la reproductibilité et éviter le data leakage. Il comprend deux branches parallèles via ColumnTransformer :

Composant	Variables	Transformation
Branche numérique	kilometer, powerPS, monthOfRegistration, age_vehicule, postalCode	SimpleImputer(médiane) + StandardScaler
Branche catégorielle	vehicleType, gearbox, model, fuelType, brand, notRepairedDamage	SimpleImputer(mode) + OneHotEncoder
Estimateur final	–	Modèle ML (XGBoost, RF, etc.)

La variable cible est transformée en $\log(1 + \text{prix})$ lors de l'entraînement, puis la prédiction finale est reconvertie via $\exp(y_{\text{pred}}) - 1$ pour obtenir le prix en EUR.

4. Modélisation et Comparaison des Algorithmes

4.1 Protocole d'Évaluation

12 algorithmes de régression sont évalués en utilisant un split 80/20 (entraînement/test) stratifié. Deux métriques principales sont retenues : le coefficient de détermination R^2 (qualité globale du fit) et le RMSE – Root Mean Squared Error (erreur en EUR, directement interprétable). Le split est effectué sur un sous-échantillon aléatoire de 50 000 lignes pour limiter les temps de calcul lors de la phase de comparaison.

4.2 Résultats de la Comparaison

Le graphique ci-dessous présente les performances comparatives des 12 modèles sur le jeu de test. Les modèles sont classés par couleur selon leurs performances : rouge ($R^2 < 0.5$), orange ($0.5 \leq R^2 < 0.85$), vert ($R^2 \geq 0.85$).

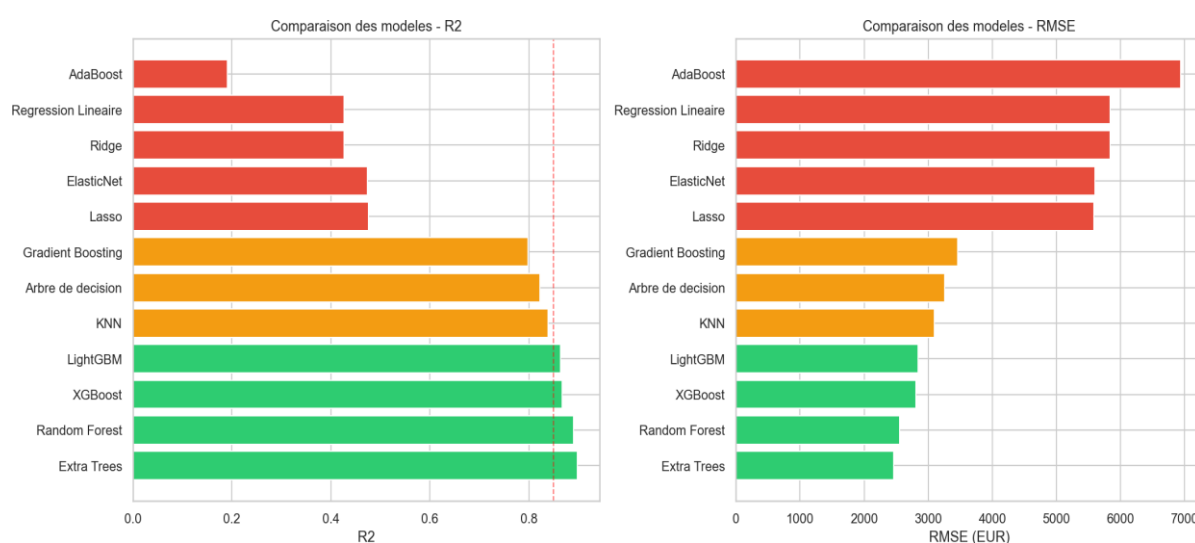


Figure 5 – Comparaison des 12 modèles : R^2 (gauche) et RMSE en EUR (droite)

Les trois modèles les plus performants sont Extra Trees ($R^2 \approx 0.87$), Random Forest ($R^2 \approx 0.86$) et XGBoost ($R^2 \approx 0.86$), tous avec un RMSE inférieur à 3 000 EUR. Les modèles linéaires (Ridge, Lasso, ElasticNet) se limitent à $R^2 \approx 0.42$, illustrant la non-linéarité des relations dans ce dataset.

5. Cross-Validation 5-Fold

5.1 Protocole

Pour valider la robustesse des modèles et détecter le surapprentissage, une cross-validation 5-fold (KFold, shuffle=True) est appliquée sur un sous-échantillon de 50 000 lignes. Cette approche partitionne les données en 5 sous-ensembles égaux : le modèle est entraîné 5 fois, chaque fois en utilisant 4 folds pour l'entraînement et 1 fold pour la validation.

5.2 Résultats

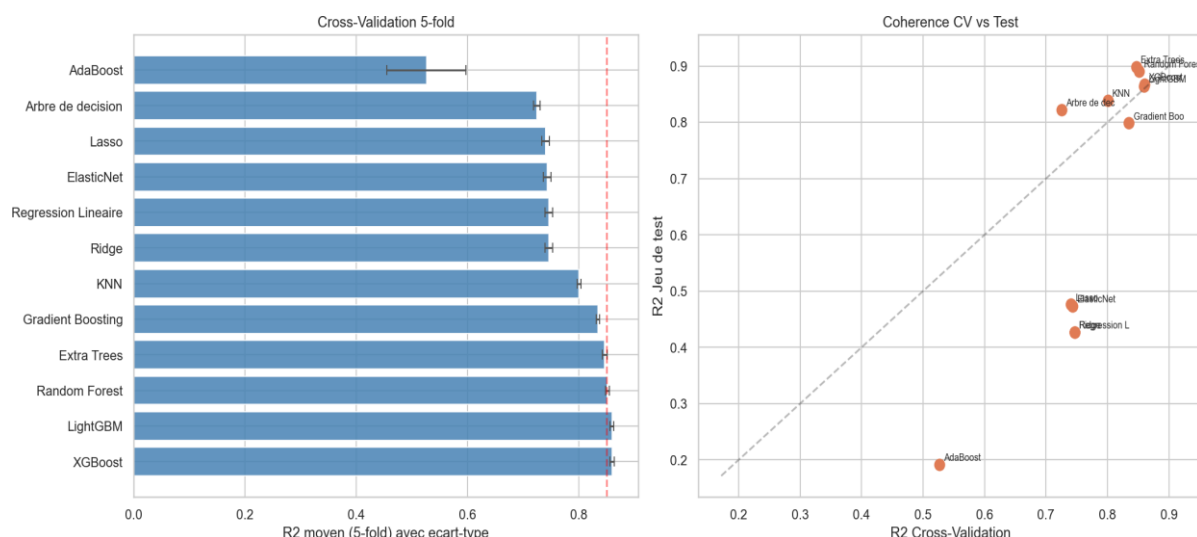


Figure 6 – R^2 moyen 5-fold avec écart-type (gauche) et cohérence CV vs Test (droite)

XGBoost obtient le meilleur R^2 CV moyen (0.862 ± 0.003), suivi de LightGBM et Random Forest. L'écart-type très faible (± 0.003) confirme la stabilité du modèle. Le graphique de cohérence CV vs Test montre que les modèles arborescents (Extra Trees, Random Forest, XGBoost, LightGBM) se situent proches de la diagonale, indiquant une bonne généralisation sans surapprentissage significatif. En revanche, les modèles linéaires présentent un R^2 CV élevé mais un R^2 test bien inférieur, révélant un problème de biais.

0.862	± 0.003	12	5
R ² CV moyen – XGBoost	Écart-type CV	Modèles évalués	Folds

6. Optimisation des Hyperparamètres

6.1 Stratégie de Tuning

Les 3 meilleurs modèles (XGBoost, LightGBM, Random Forest) sont soumis à un tuning des hyperparamètres via RandomizedSearchCV (20 itérations, validation croisée 3-fold). Cette approche explore aléatoirement l'espace des hyperparamètres, offrant un bon compromis entre qualité de la recherche et temps de calcul.

6.2 Espaces de Recherche

Modèle	Hyperparamètres explorés
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight
LightGBM	n_estimators, max_depth, learning_rate, num_leaves, subsample, colsample_bytree
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features

6.3 Impact du Tuning

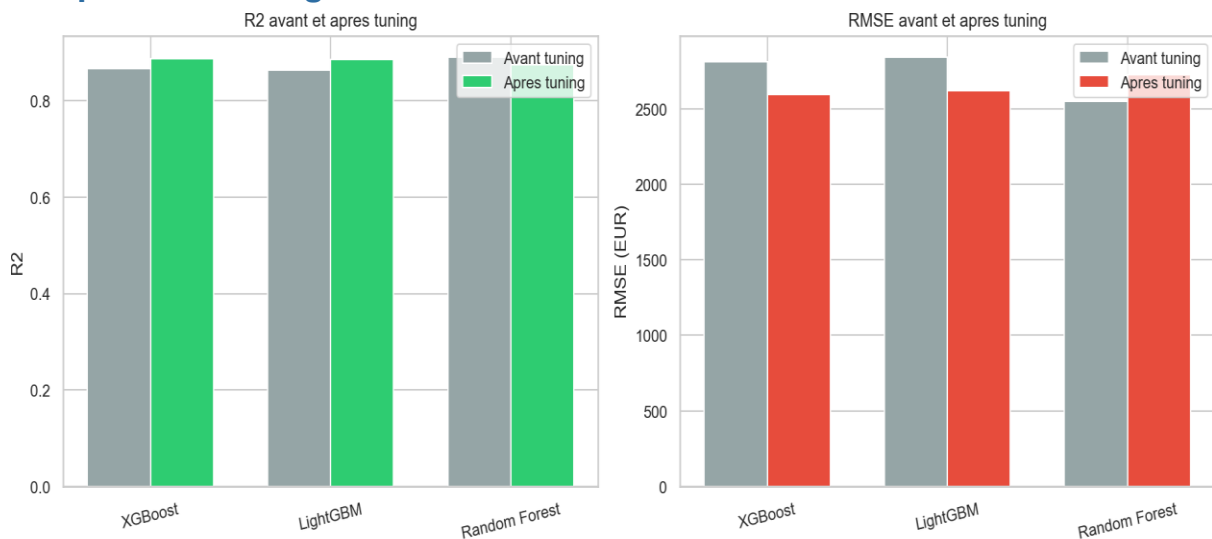


Figure 7 – Comparaison R^2 et RMSE avant/après tuning pour les 3 meilleurs modèles

Le tuning améliore modestement mais systématiquement les performances. XGBoost bénéficie de la plus grande amélioration, passant d'un R^2 de ~0.87 à 0.889, avec une réduction du RMSE de ~2 800 EUR à 2 595 EUR. LightGBM et Random Forest montrent des gains similaires. XGBoost est sélectionné comme modèle final en raison de ses meilleures performances et de sa robustesse.

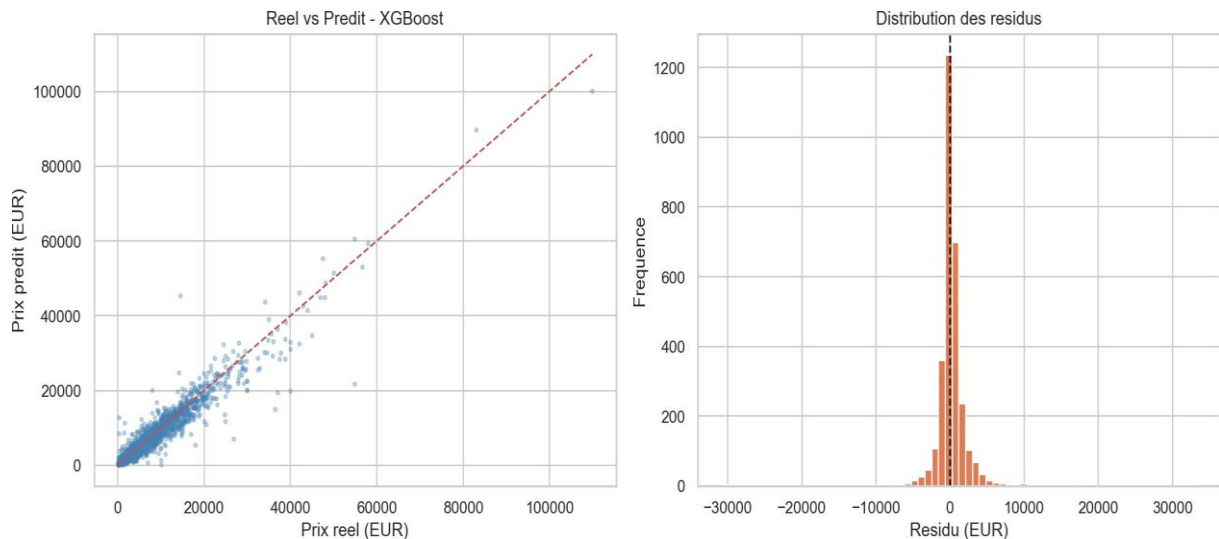
7. Évaluation Finale du Modèle XGBoost

7.1 Métriques de Performance

0.887	2 595 €	1 184 €	0.862
R ² Test	RMSE	MAE	R ² CV moyen

7.2 Analyse des Résidus et Prédictions

Le graphique Prix Réel vs Prix Prédit confirme la qualité du modèle : les points s'alignent étroitement autour de la diagonale idéale ($y = x$), avec une légère dispersion pour les véhicules haut de gamme (> 40 000 EUR) où les données sont plus rares. La distribution des résidus est centrée sur 0 et approximativement normale, indiquant l'absence de biais systématique.



7.3 Importance des Variables

L'analyse de l'importance des features révèle que **l'âge du véhicule** est de loin la variable la plus prédictive (importance ≈ 0.09), suivie de la marque Porsche (effet premium fort). Parmi les variables d'origine, powerPS (puissance) et kilometer (kilométrage) complètent le top 5. La présence de dommages non réparés (notRepairedDamage_ja) a un impact significatif négatif sur le prix.

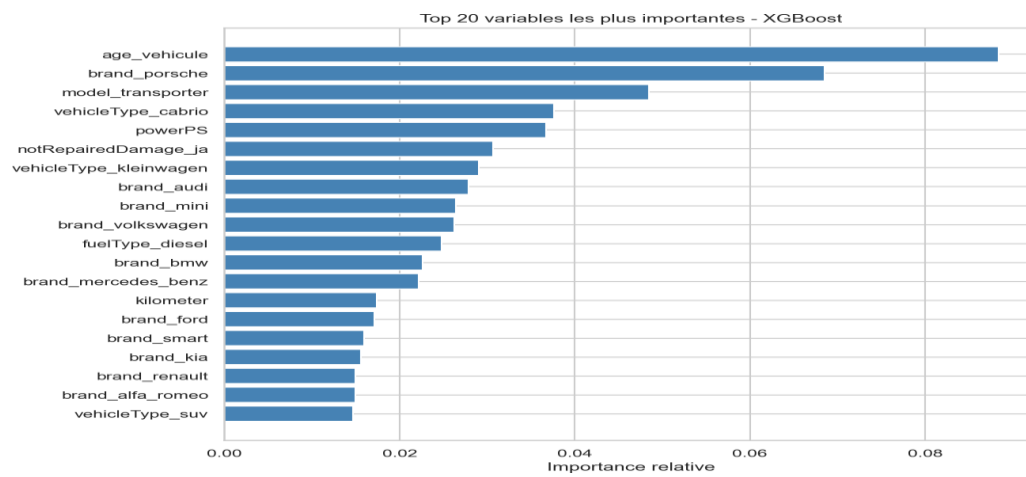


Figure 9 – Top 20 variables les plus importantes (XGBoost – importance relative)

8. Sérialisation et Déploiement

8.1 Sérialisation du Pipeline

L'intégralité du pipeline (preprocessing + modèle XGBoost) est sérialisée avec joblib pour un déploiement immédiat :

- **car_price_pipeline.joblib** – Pipeline sklearn complet (preprocessing + XGBoost tuné)
- **model_metadata.json** – Métadonnées du modèle (métriques, listes de valeurs valides pour chaque feature catégorielle)

8.2 Application Streamlit

Une application web interactive est développée avec Streamlit (app.py). Elle permet à tout utilisateur de saisir les caractéristiques d'un véhicule et d'obtenir instantanément une estimation du prix de vente. L'interface inclut les composants suivants :

- **Sélecteurs marque/modèle** : Listes déroulantes dynamiques basées sur les valeurs du modèle
- **Curseurs numériques** : Kilométrage (0–300 000 km), Puissance (50–500 CV), Année
- **Sélecteurs catégoriels** : Type de carrosserie, Boîte de vitesse, Carburant, Dommages
- **Résultat en temps réel** : Prix estimé affiché avec fourchette de confiance
- **Informations techniques** : Métriques du modèle affichées dans la sidebar

8.3 Structure des Fichiers

Fichier	Description
notebook.ipynb	Pipeline ML complet (EDA → Déploiement)
app.py	Application Streamlit de prédiction
car_price_pipeline.joblib	Pipeline sérialisé (preprocessing + XGBoost)
model_metadata.json	Métadonnées et valeurs valides
requirements.txt	Dépendances Python
Rapport_Projet.pdf	Ce rapport

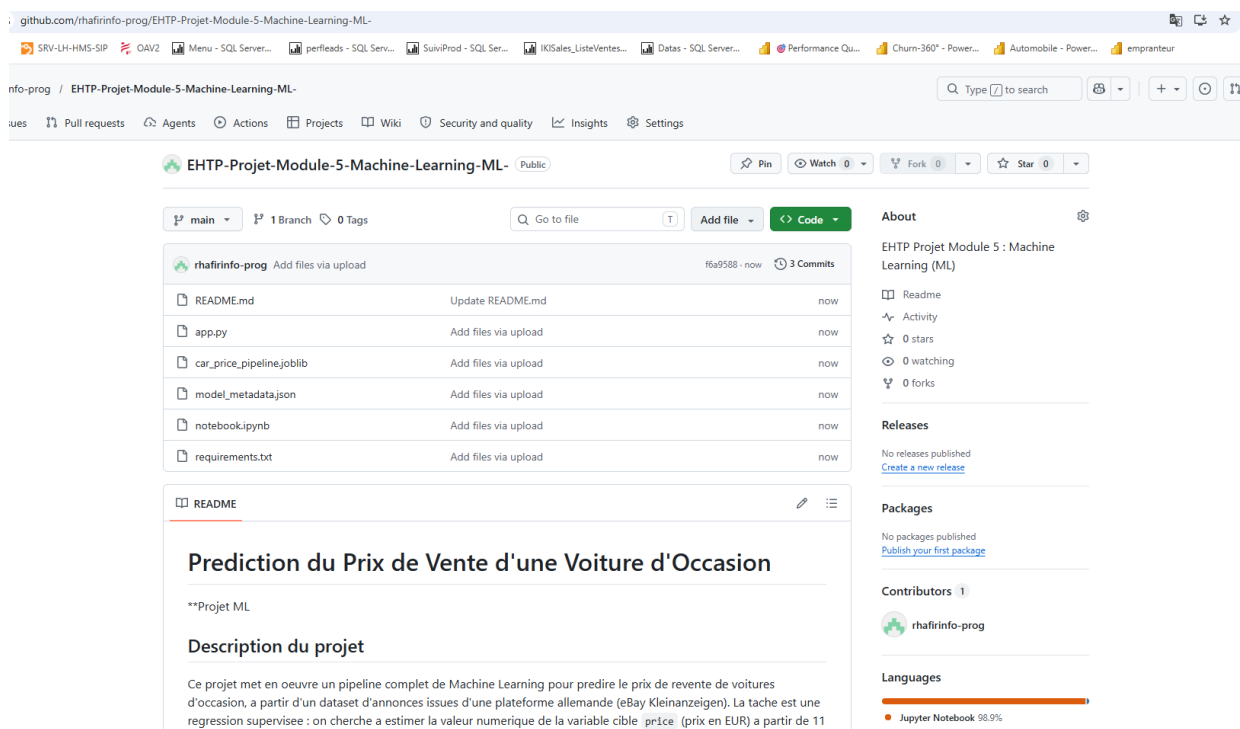
9. Conclusion

9.1 Bilan du Projet

Ce projet a permis de mettre en œuvre un pipeline ML end-to-end complet, depuis l'acquisition et l'analyse des données jusqu'au déploiement d'une application web fonctionnelle. Le modèle XGBoost retenu atteint des performances solides ($R^2 = 0.887$, RMSE = 2 595 EUR) sur un dataset réel et volumineux. L'âge du véhicule, la marque et la puissance s'avèrent être les déterminants les plus importants du prix de revente.

9.2 GitHub

Lien : <https://github.com/rhafirinfo-prog/EHTP-Projet-Module-5-Machine-Learning-ML->



9.3 Streamlit

Lien : <https://youssefrhafir.streamlit.app/>

